

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Deep Learning

COURSE CODE: MLA04

COURSE OUTCOME COVERED

CO1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BL3)

QUESTIONS: 1

Duration: 1 Hour
Total Marks:50

Annexure A

Error Analysis Task

1. A neural network predicts student scores with an MSE of 0.85 during training and 5.20 during testing. Analyse the possible reasons for this difference and suggest suitable corrective measures.

Code of Conduct:

I Haresh Kumar N L (192425009) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5)	Level 3 – Proficient (4)	Level 2 – Developing (2-3)	Level 1 – Beginning (0-1)
Error Identification	30%	Correctly identifies all errors and their causes	Identifies most errors with minor omissions	Identifies some errors but misses key issues	Unable to identify errors
Error Analysis & Reasoning	30%	Provides clear and logical explanation of why errors occurred	Reasoning mostly corrects with minor gaps	Limited reasoning and partial explanation	No meaningful analysis
Correction Strategy	25%	Suggests appropriate corrections with justification	Suggests correct corrections with minor gaps	Partially correct corrections	Incorrect or no corrections
Interpretation of Results	15%	Clearly explains impact of errors on model performance/results	Basic interpretation with minor omissions	Limited interpretation	No interpretation provided

Answer

1) A neural network is trained to predict student scores and achieves:

- **Training Mean Squared Error (MSE) = 0.85**
- **Testing Mean Squared Error (MSE) = 5.20**

The **training MSE** is very low, indicating that the model fits the training data well. However, the **testing MSE** is much higher, showing that the model performs poorly on unseen data. This significant difference suggests that the neural network has **overfitted** the training dataset. Overfitting occurs when a model learns not only the underlying patterns in the data but also the noise and random variations, reducing its ability to generalize to new data.

Possible Reasons for the Difference

1. **Overfitting**
 - The model has memorized the training data instead of learning general patterns.
 - As a result, it achieves low error on the training set but high error on the testing set.
2. **Complex Neural Network Architecture**
 - The network may contain too many hidden layers or neurons for the given dataset.
 - A highly complex model can capture unnecessary details and noise, leading to poor generalization.
3. **Insufficient Training Data**
 - If the training dataset is too small or lacks diversity, the model cannot learn representative patterns.
 - This causes the model to perform poorly when tested on new data.
4. **Noisy or Poor-Quality Data**
 - The training data may contain incorrect labels, missing values, or outliers.
 - The model may learn these unwanted patterns, increasing the testing error.
5. **Lack of Regularization**
 - Regularization techniques such as **Dropout**, **L1**, or **L2 Regularization** may not have been applied.
 - Without regularization, the model becomes overly dependent on the training data.
6. **Improper Hyperparameter Selection**
 - An unsuitable learning rate, excessive number of training epochs, or an inappropriate batch size can lead to overfitting.
 - Training the model for too many epochs allows it to memorize the training data instead of learning generalized features.
7. **Data Distribution Difference**
 - The training and testing datasets may have different characteristics or distributions.
 - This causes the model to struggle when making predictions on the test data.

Corrective Measures

1. **Apply Regularization Techniques**
 - Use **L1/L2 Regularization** to penalize large weights.
 - Apply **Dropout** to randomly deactivate neurons during training, reducing overfitting.
2. **Increase the Training Dataset**
 - Collect more training samples or use **data augmentation** techniques.
 - A larger and more diverse dataset helps the model learn better general patterns.

3. **Simplify the Neural Network**
 - Reduce the number of hidden layers or neurons.
 - A simpler model is less likely to memorize the training data.
4. **Use Early Stopping**
 - Monitor the validation error during training.
 - Stop training when the validation error starts increasing to prevent overfitting.
5. **Perform Cross-Validation**
 - Use techniques such as **k-fold cross-validation** to evaluate the model on different subsets of data.
 - This provides a more reliable estimate of model performance.
6. **Tune Hyperparameters**
 - Experiment with different learning rates, batch sizes, activation functions, and optimization algorithms.
 - Proper tuning improves both learning efficiency and generalization.
7. **Improve Data Quality**
 - Remove outliers, handle missing values, and correct mislabeled data.
 - Clean data enables the model to learn meaningful patterns.
8. **Monitor Validation Performance**
 - Use a separate validation dataset during training.
 - Compare training, validation, and testing errors to detect overfitting early.

Conclusion

The large gap between the **training MSE (0.85)** and **testing MSE (5.20)** clearly indicates that the neural network is **overfitting**. Although the model performs exceptionally well on the training data, it fails to generalize to unseen test data. This issue can be addressed by applying regularization techniques, increasing the amount of training data, simplifying the network architecture, using early stopping, performing cross-validation, tuning hyperparameters, and improving data quality. These measures will help reduce the testing error and produce a more accurate and reliable neural network model.